

CampusMesh

Internet-Independent Peer-to-Peer Campus Communication Network

Project Report

Programming in Java

Submitted by

Suryansh Pratap Singh

Registration No.: 25BAI10845

CSE (AI & ML) — 2nd Year

VIT Bhopal University

Abstract

It's easy to assume a message will simply reach whoever you send it to. On a large campus, that assumption falls apart fast — a dead zone in the hostel wing, a router that gives up during exam week, or just too many phones fighting over the same Wi-Fi channel. CampusMesh grew out of a fairly simple question I kept coming back to: if the internet vanished for an hour, could the devices around me still talk to each other without it? What came out of that is a Java prototype where devices form a small network of their own, passing messages from one to the next until they land where they're supposed to.

Along the way, the project picked up a few abilities I hadn't planned for at the start — rerouting around a broken link, holding a message for a recipient who's stepped offline, sounding an alert to everyone nearby during an emergency, and checking that a transferred file actually arrived the way it left. None of this depends on an internet connection existing at all.

1. Introduction

Anyone who has lived on a college campus knows connectivity is never quite as solid as it should be. Coverage gaps, the occasional outage, and plain old congestion during peak hours can turn something as ordinary as messaging someone two buildings away into a small ordeal. In an emergency, that stops being a minor annoyance — it becomes a real problem, and a fairly urgent one.

CampusMesh is a modest attempt at fixing that. Rather than routing everything through one central network, every device that joins becomes a node capable of passing messages on to its neighbours. When two devices can't reach each other directly, whatever devices happen to sit between them simply relay the message forward — so the network as a whole keeps working even if no single link in it is especially reliable.

2. Objectives

Going in, the goals were fairly concrete:

- Build an actual working peer-to-peer system — not just something that looks good on paper.
- Let devices discover one another and form a network of nodes on their own.
- Route messages across multiple hops whenever a direct link isn't available.
- Hold messages for recipients who are offline and deliver them the moment they reconnect.
- Give the network a way to broadcast urgent SOS alerts to everyone it can reach.
- Move files between devices with some way to confirm nothing got corrupted in transit.
- Keep a log of network activity for debugging and for reviewing what happened later.
- Use the project as a genuine opportunity to apply core Java ideas — OOP, multithreading, exception handling — instead of leaving them as textbook material.

3. Technologies Used

Technology	Purpose
Java	Core programming language
Maven	Build and dependency management
TCP/IP, ServerSocket	Local network communication
Multithreading	Concurrent connection handling
BFS	Multi-hop route discovery
SHA-256	File integrity verification
CLI	User interaction

4. System Architecture

Rather than cramming everything into one bloated class, the system is split into layers, each with one job to do. A command-line interface takes in whatever the user types. An application layer works out what that command actually means. A services layer does the real work — sending messages, moving files, syncing state, computing routes. And underneath all of it sits the peer-to-peer network itself: just a collection of nodes talking to their neighbours.

Command Line UI → Application Layer → Network / Services (Messaging, Files, Sync, Routing) → Peer-to-Peer Network (Node ↔ Node ↔ Node)

5. Core Modules

5.1 Multi-Hop Routing

This is really the heart of the whole project. When two devices can't talk to each other directly, the router treats the network as a graph and runs a Breadth-First Search to find the shortest chain of hops between them. That's how a message from A can still reach D by passing through B and C, even though A and D have never once been in range of each other.

5.2 Store-and-Forward Messaging

Devices go offline constantly — phones die, laptops sleep, people simply walk out of range. Instead of dropping the message when that happens, the system holds onto it until the recipient shows up again on the network, at which point it gets delivered automatically. It's a small feature on paper, but it's really what makes the network feel dependable instead of fragile.

5.3 SOS / Emergency Alerts

During an emergency, waiting on a routed, one-to-one message just isn't good enough — you want the alert reaching as many people as possible, as fast as possible. That's exactly what the SOS command is for: it pushes a message out to every node the network can currently reach, all at once.

5.4 File Transfer with Integrity Verification

Sending a file is the easy part; trusting that it arrived in one piece is what's harder. Before a file leaves the sender, it gets run through SHA-256 to produce a hash. Once it arrives, that same hash is computed again on the receiving end — if the two match, nothing was lost or altered along the way.

5.5 Multithreaded Networking

A server that can only handle one device at a time isn't really much of a network. So every incoming connection here gets handed off to its own worker thread, which means the server can keep several conversations running at once without one slow connection holding everything else up.

6. Implementation Snapshot

Once it's built with Maven, the whole thing runs as a command-line application — there's no GUI to speak of, on purpose. That keeps the focus on the network logic itself rather than on building an interface around it.

mvn clean package | mvn exec:java

A few of the commands you'd actually reach for day-to-day:

Command	Function
addnode / connect	Build the network topology
route	Compute a path between two nodes
send / sos	Deliver a normal or emergency message
file	Transfer a file with checksum verification
sync / logs / status	Synchronize state and inspect system activity

7. Testing Summary

The system was put through a set of scenarios that covered what it's actually meant to handle — not just the happy path, but the failure cases too:

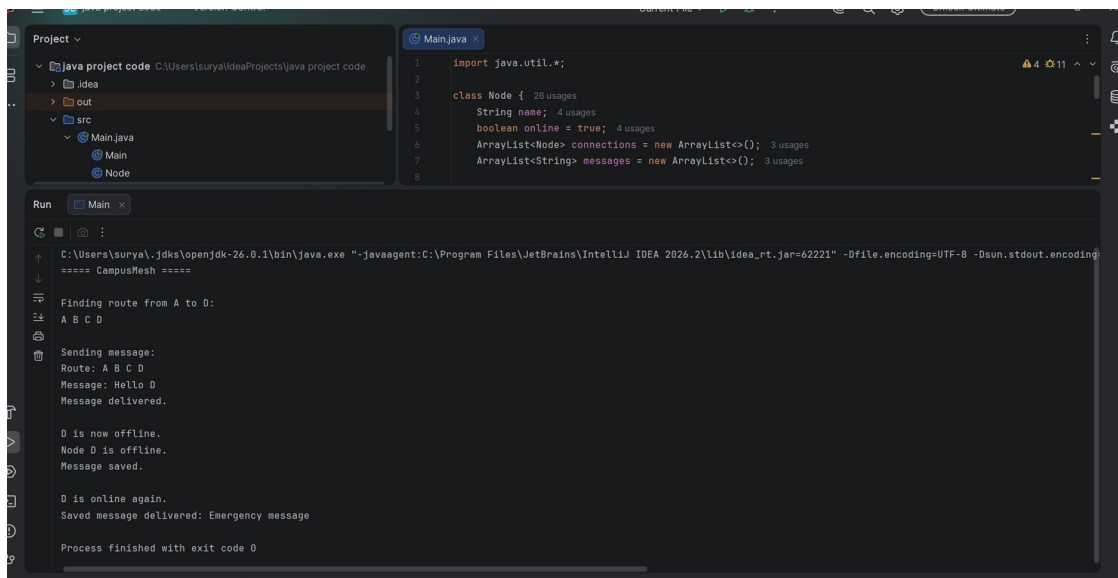
Test Case	Result
Direct message delivery	Message reaches destination
Multi-hop message	Message follows computed route
Offline destination	Message stored and later delivered
File transfer	File transferred and checksum verified
SOS broadcast	Alert reaches all reachable nodes
Node / server failure	Handled gracefully without crash

8. Output

The screenshot below is taken directly from a run of the program inside IntelliJ IDEA, and it walks through most of the core modules in one pass rather than in isolation. It's worth reading through, since it shows the system actually doing the things described above rather than just being described.

The run starts by asking the router to find a path from node A to node D. Since the two aren't directly connected, the BFS-based router works out the route $A \rightarrow B \rightarrow C \rightarrow D$, and the console prints that chain before anything is sent. A message ("Hello D") is then sent along that exact route and confirmed as delivered — the multi-hop routing described in Section 5.1 working end to end.

Node D is then taken offline. Rather than the message being lost, the store-and-forward mechanism from Section 5.2 kicks in: the log confirms the node is offline and that the outgoing message has been saved rather than dropped. When D reconnects ("D is online again"), the saved message — in this run, an emergency message — is delivered automatically, with no manual intervention needed to trigger it. The program exits cleanly with code 0, which is the expected, non-crashing termination for this session.



```
1 import java.util.*;
2
3 class Node {
4     String name;
5     boolean online = true;
6     ArrayList<Node> connections = new ArrayList<>();
7     ArrayList<String> messages = new ArrayList<>();
8 }
```

```
==== CampusMesh ====
Finding route from A to D:
A B C D

Sending message:
Route: A B C D
Message: Hello D
Message delivered.

D is now offline.
Node D is offline.
Message saved.

D is online again.
Saved message delivered: Emergency message

Process finished with exit code 0
```

Fig. 1 — Console output from IntelliJ IDEA showing routing, message delivery, offline storage, and delayed delivery upon reconnection.

9. Limitations and Future Work

It's worth being honest about what this project isn't. Right now, the peer-to-peer behaviour is simulated over local TCP/IP — there's no actual Bluetooth or Wi-Fi Direct discovery happening between real, separate devices yet. That's the obvious next step, along with an Android build so it could realistically run on the phones people already carry around campus. Beyond that, encryption, smarter weighted routing, and even long-range options like LoRa all seem like reasonable directions if this were pushed further.

10. Conclusion

What this project shows, at least at prototype scale, is that you don't actually need the internet to build something useful for campus communication. Peer-to-peer networking, routing that adapts around broken links, messages that wait patiently for offline recipients, and file transfers you can trust — put together, they add up to something small but genuinely resilient. Just as important, building it meant putting object-oriented design, concurrency, and graph algorithms to real use, rather than leaving them as ideas confined to a textbook.

Project Details

Field	Detail
Name	Suryansh Pratap Singh
Registration No.	25BAI10845
Branch	CSE AI & ML
Year	2nd Year
Institution	VIT Bhopal University
Course	Programming in Java